Analisador de Programas Pascal Simplificado

Analisador léxico, sintático e semântico (parcial) para uma linguagem do tipo Pascal, escrito em Haskell com Alex e Happy. O programa lê o código do stdin, verifica se todas as variáveis usadas foram declaradas e, se não houver erros de sintaxe, imprime a árvore sintática abstrata (AST) num formato indentado.

Estrutura do projeto

`Lexer.x` - Analisador léxico. Ignora comentarios com as primeiras duas expressoes regulares e ignora maiusculas e minusculas de palavras chave. Converte o texto em *tokens*.

`Parser.y` - Analisador sintático (Happy) em formato de gramática que constrói a árvore a partir dos tokens.

`AST.hs` - Definição dos tipos de dados da árvore sintática abstrata.

`Check.hs` - Análise semântica que verifica o uso de variáveis não declaradas.

`Pretty.hs` - Impressão da AST como árvore indentada.

`Main.hs` - Ponto de entrada: liga todas as fases.

Compilação

Requisitos: GHC, Alex e Happy.

```
alex Lexer.x       # gera Lexer.hs
happy Parser.y     # gera Parser.hs
ghc Main.hs -o analisador
```

Utilização

```
./analisador < programa.pas
```

- Se o programa estiver correto, a AST é escrita no `stdout`.
- Se existirem variáveis não declaradas, os erros são escritos no `stderr` (prefixados por `Erro:`) e o programa termina com código de saída diferente de zero.
- Erros de sintaxe provocam uma mensagem `Erro de sintaxe perto de: ...` com os próximos *tokens*.

Comandos

- Atribuição: `x := expressão` (numérica/string ou booleana)
- Bloco: `begin ... end`
- Condicional: `if cond then cmd [else cmd]` (o `else` liga-se ao `if` mais próximo)
- Ciclo: `while cond do cmd`
- Escrita: `write`, `writeln`, com ou sem argumentos (`write(a, b, 'texto')`)

- Leitura: `read(a, b)`, `readln(a, b)`

Expressões

- Numéricas: `+`, `-`, `*`, `/`, `div`, `mod`, negação unária, parênteses
- Literais: inteiros, reais (`3.14`), *strings* (`'texto'`)
- Booleanas: `or`, `and`, `not`, `true`, `false` e comparações (`=`, `<>`, `<`, `>`, `<=`, `>=`)

Precedência (da menor para a maior)

`or` < `and` < comparadores (não associativos) < `+ -` < `* / div mod` < `not` / `-` unário

Aspetos léxicos

- Palavras reservadas **insensíveis a maiúsculas/minúsculas** (`BEGIN`, `Begin`, `begin`...).
- Comentários de linha com `//` e comentários de bloco com `{ ... }`.
- Espaços em branco são ignorados.

Análise semântica

Atualmente `Check.hs` constrói uma tabela de símbolos (`nome → tipo`) a partir das declarações e verifica que toda a variável usada (em atribuições, expressões, `read` e `write`) foi declarada. A verificação de tipos fica reservada para uma fase posterior.